



INSTITUTO DE
INVESTIGACIONES
EN MATEMÁTICAS
APLICADAS Y
EN SISTEMAS

Universidad Nacional Autónoma de México
Instituto de Investigaciones en Matemáticas Aplicadas y en Sistemas

Análisis de Sentimientos con Redes Neuronales Recurrentes

Tarea

Materia:	Aprendizaje Profundo
Profesor:	Zeus Tlaltecutli Domínguez Vega
Ayudante:	Luz Citlalli Jiménez Terrazas
Semestre:	8o. Semestre

Alumno

Carlos Emiliano Mendoza Hernández

Ciudad Universitaria, 17 de mayo de 2026

Índice

1. Introducción	3
2. Marco Teórico	3
2.1. Notación	3
2.2. Long Short-Term Memory (LSTM)	4
2.3. Gated Recurrent Unit (GRU)	4
2.4. Redes Recurrentes Apiladas (Stacked RNN)	5
3. Arquitectura del Modelo	6
3.1. Capa de Embedding	6
3.2. Capas Recurrentes Apiladas	6
3.3. Estrategia de Regularización	6
3.4. Capa de Clasificación	6
3.5. Inicialización del Bias de la Compuerta de Olvido	7
3.6. Resumen de la Arquitectura	7
3.7. Implementación en Código	7
4. Metodología de Entrenamiento	10
4.1. Preprocesamiento de Datos	10
4.2. Función de Pérdida y Optimizador	11
4.3. Bucle de Entrenamiento	12
4.4. Gradient Clipping	12
4.5. Early Stopping	13
4.6. Reproducibilidad y Configuración	13
5. Resultados Experimentales	15
5.1. Métricas de Clasificación	15
5.2. Tiempos de Entrenamiento	15
5.3. Curvas de Entrenamiento	15
5.4. Matrices de Confusión	16
5.5. Análisis de Normas de Gradiente	16
5.6. Proceso de Inferencia	17
6. Análisis de Gradientes	19
6.1. Normas Promedio de Gradientes	19
6.2. Ausencia de Vanishing y Exploding Gradients	19
6.3. Rol de las Compuertas en la Estabilidad de Gradientes	19
6.4. Normas de Gradiente por Capa	20
6.5. Visualización del Flujo de Gradientes	22
7. Reflexiones y Discusión	23
7.1. Efecto del número de capas apiladas en la capacidad de aprendizaje	23
7.2. Impacto del dropout entre capas RNN en la generalización	23
7.3. Comparación de velocidad de convergencia LSTM vs GRU	24

7.4. Diferencias en el flujo de gradientes entre LSTM y GRU	24
7.5. Efecto de la longitud de secuencia (padding/truncamiento) en el rendimiento	24
7.6. Papel de la capa de embedding en la representación del sentimiento	25
7.7. Efectividad del early stopping en la prevención del sobreajuste	25
7.8. Trade-offs entre LSTM y GRU para análisis de sentimiento	26
7.9. Estabilidad numérica de BCEWithLogitsLoss frente a Sigmoid + BCELoss	26
7.10. Análisis de falsos positivos y su impacto en Precision y Recall	27
7.11. Conclusión general	28
8. Código Fuente	29
8.1. Definición del Modelo	29
8.2. Entrenamiento	31
8.3. Preprocesamiento de Datos	33
8.4. Evaluación e Inferencia	33
9. Referencias	35

1. Introducción

El análisis de sentimientos es una tarea fundamental del procesamiento de lenguaje natural que consiste en determinar la polaridad emocional de un texto. En este trabajo abordamos el problema de **clasificación binaria de sentimiento**: dada una reseña de película del dataset IMDB, el objetivo es clasificarla como *positiva* o *negativa*. Este dominio resulta particularmente interesante porque las reseñas contienen dependencias de largo alcance, negaciones, sarcasmo y estructuras sintácticas complejas que requieren un modelado secuencial adecuado. Las redes neuronales recurrentes (RNN) son la familia de arquitecturas naturalmente diseñada para procesar secuencias de longitud variable, ya que mantienen un estado oculto que se actualiza en cada paso temporal incorporando la información del token actual y del contexto previo. Sin embargo, las RNN simples sufren del problema de *vanishing gradients*, lo que dificulta el aprendizaje de dependencias a largo plazo. Las arquitecturas LSTM [1] y GRU [2] resuelven este problema mediante mecanismos de compuertas que regulan el flujo de información a través del tiempo.

En este reporte presentamos un estudio comparativo entre arquitecturas LSTM y GRU apiladas (*Stacked RNN*) aplicadas a la clasificación de sentimiento en IMDB, analizando su rendimiento, velocidad de convergencia y estabilidad del flujo de gradientes.

2. Marco Teórico

2.1. Notación

A lo largo de este documento utilizamos la siguiente notación. Sea $x_t \in \mathbb{R}^d$ el vector de entrada en el paso temporal t (proveniente de la capa de embedding) y $h_{t-1} \in \mathbb{R}^h$ el estado oculto del paso anterior, donde d es la dimensión de entrada y h la dimensión oculta. Definimos:

- $W \in \mathbb{R}^{h \times d}$: matriz de pesos que transforma la entrada x_t .
- $U \in \mathbb{R}^{h \times h}$: matriz de pesos que transforma el estado oculto previo h_{t-1} .
- $b \in \mathbb{R}^h$: vector de bias.
- $\sigma(\cdot)$: función sigmoide, $\sigma(z) = \frac{1}{1+e^{-z}}$, que produce valores en $(0, 1)$ utilizados como compuertas.
- $\tanh(\cdot)$: tangente hiperbólica, que produce valores en $(-1, 1)$ utilizados para candidatos de estado.
- $\odot$: producto elemento a elemento (producto de Hadamard).

Los subíndices en las matrices (W_f , U_i , b_o , etc.) indican a qué compuerta pertenece cada conjunto de parámetros.

2.2. Long Short-Term Memory (LSTM)

La celda LSTM [1] introduce tres compuertas y un estado de celda c_t que permite el flujo de gradientes a largo plazo. Las ecuaciones que gobiernan su dinámica son las siguientes.

La **compuerta de olvido** determina qué información del estado de celda anterior se descarta:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (1)$$

La **compuerta de entrada** controla qué nueva información se almacena en el estado de celda:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (2)$$

El **candidato de celda** genera los valores candidatos que podrían agregarse al estado:

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad (3)$$

El **estado de celda** se actualiza combinando el olvido selectivo del estado anterior con la nueva información filtrada por la compuerta de entrada:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (4)$$

Finalmente, la **compuerta de salida** y el **estado oculto** determinan qué parte del estado de celda se expone como salida:

$$\begin{aligned} o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\ h_t &= o_t \odot \tanh(c_t) \end{aligned} \quad (5)$$

El mecanismo clave del LSTM es el estado de celda c_t (Ecuación (4)), que actúa como una “autopista de información” donde los gradientes pueden fluir sin degradarse, gracias a que la compuerta de olvido (1) permite mantener información relevante de pasos temporales lejanos.

2.3. Gated Recurrent Unit (GRU)

La GRU [2] simplifica la arquitectura LSTM eliminando el estado de celda separado y utilizando únicamente dos compuertas. A pesar de tener menos parámetros, logra un rendimiento comparable en muchas tareas.

La **compuerta de reset** determina cuánto del estado oculto anterior se ignora al calcular el candidato:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (6)$$

La **compuerta de actualización** controla el balance entre mantener el estado anterior y adoptar el nuevo candidato:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (7)$$

El **candidato oculto** se calcula aplicando la compuerta de reset al estado anterior:

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad (8)$$

El **estado oculto** se obtiene como interpolación lineal entre el estado anterior y el candidato, controlada por la compuerta de actualización:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (9)$$

Nótese que cuando $z_t \approx 0$ en la Ecuación (9), el estado se preserva sin cambios (análogo a la compuerta de olvido del LSTM en (1)), mientras que cuando $z_t \approx 1$, el estado se reemplaza completamente por el candidato (8). La compuerta de reset (6) permite al modelo “olvidar” selectivamente el pasado al calcular el nuevo candidato.

2.4. Redes Recurrentes Apiladas (Stacked RNN)

Una *Stacked RNN* consiste en apilar múltiples capas recurrentes de forma que la salida de una capa sirve como entrada para la siguiente. Formalmente, si denotamos por $h_t^{(l)}$ el estado oculto de la capa l en el paso temporal t , entonces:

$$h_t^{(l)} = \text{RNN}^{(l)}(h_t^{(l-1)}, h_{t-1}^{(l)}) \quad (10)$$

donde $h_t^{(0)} = x_t$ es la entrada original (el embedding del token) y $\text{RNN}^{(l)}$ puede ser una celda LSTM (Ecuaciones (1)–(5)) o GRU (Ecuaciones (6)–(9)).

El apilamiento de capas permite que la red aprenda **representaciones jerárquicas** del texto: las capas inferiores capturan patrones locales y sintácticos (n-gramas, estructura de frases), mientras que las capas superiores combinan estas representaciones para extraer semántica de mayor nivel de abstracción (sentimiento global, relaciones entre cláusulas). Esta capacidad de abstracción progresiva es análoga a la que exhiben las redes convolucionales profundas en visión por computadora, donde las capas iniciales detectan bordes y las superiores reconocen objetos completos.

En nuestro experimento utilizamos $L = 2$ capas apiladas (véase Ecuación (10)), lo que proporciona un balance entre capacidad representacional y eficiencia computacional.

3. Arquitectura del Modelo

El modelo `SentimentRNN` es una red neuronal recurrente apilada configurable, diseñada para clasificación binaria de sentimiento. Su arquitectura soporta tanto celdas LSTM como GRU como tipo de capa recurrente, permitiendo una comparación directa entre ambas variantes bajo condiciones idénticas. A continuación se describe cada componente de la arquitectura, cuyo resumen se presenta en la Tabla 1.

3.1. Capa de Embedding

La primera capa del modelo es un embedding que transforma cada token del vocabulario en un vector denso de representación distribuida. El vocabulario tiene un tamaño de 25,000 tokens y cada uno se proyecta a un espacio de dimensión 128. Esta capa contiene $25,000 \times 128 = 3,200,000$ parámetros entrenables y constituye la mayor parte de los parámetros del modelo.

3.2. Capas Recurrentes Apiladas

El modelo utiliza 2 capas recurrentes apiladas, cada una con 256 unidades ocultas. La salida de la primera capa recurrente (secuencia de estados ocultos) alimenta la entrada de la segunda capa, permitiendo al modelo aprender representaciones jerárquicas de mayor nivel de abstracción. Dependiendo de la configuración, las capas pueden ser LSTM o GRU, manteniendo una interfaz idéntica para ambas variantes.

3.3. Estrategia de Regularización

Para prevenir el sobreajuste, el modelo implementa una estrategia de regularización basada en dropout en dos puntos:

- **Dropout post-embedding** ($p = 0.5$): aplicado inmediatamente después de la capa de embedding, antes de alimentar la secuencia a las capas recurrentes. Este dropout agresivo actúa como una forma de aumento de datos a nivel de representación.
- **Dropout entre capas RNN** ($p = 0.3$): aplicado entre las capas recurrentes apiladas mediante el parámetro `dropout` del módulo RNN de PyTorch. Este regulariza las representaciones intermedias entre capas.

3.4. Capa de Clasificación

La capa final es una transformación lineal que recibe el estado oculto de la última capa recurrente en el último paso temporal, con dimensión $256 \rightarrow 1$. La salida es un logit escalar que, al aplicarle la función sigmoide, produce la probabilidad de que la reseña sea positiva. Esta capa contiene $256 + 1 = 257$ parámetros (pesos más bias).

3.5. Inicialización del Bias de la Compuerta de Olvido

Para las variantes LSTM, el bias de la compuerta de olvido (*forget gate*) se inicializa a 1.0 siguiendo la recomendación de [3]. Esta inicialización favorece que la celda LSTM recuerde información desde las primeras etapas del entrenamiento, evitando que el gradiente se desvanezca prematuramente a través de la compuerta de olvido antes de que el modelo haya aprendido qué información retener.

3.6. Resumen de la Arquitectura

Cuadro 1: Arquitectura completa del modelo `SentimentRNN`. Se muestran las dimensiones de entrada y salida de cada capa, así como el número de parámetros entrenables para las variantes LSTM y GRU.

Capa	Tipo	Entrada	Salida	Parámetros
Embedding	<code>nn.Embedding</code>	25,000	128	3,200,000
Dropout (embed)	<code>nn.Dropout(0.5)</code>	128	128	0
RNN Capa 1	LSTM/GRU	128	256	LSTM: 394,240 / GRU: 295,680
RNN Capa 2	LSTM/GRU	256	256	LSTM: 525,312 / GRU: 393,984
Lineal	<code>nn.Linear</code>	256	1	257

3.7. Implementación en Código

A continuación se presentan los fragmentos clave de la implementación en `model.py` que materializan la arquitectura descrita.

El constructor de la clase `SentimentRNN` define las capas del modelo: embedding, dropout, RNN apilada y capa lineal de salida. La selección entre LSTM y GRU se realiza dinámicamente mediante el parámetro `rnn_type`.

```

1  class SentimentRNN(nn.Module):
2      def __init__(
3          self,
4          vocab_size: int,
5          embedding_dim: int,
6          hidden_size: int,
7          num_layers: int,
8          rnn_type: str,
9          rnn_dropout: float,
10         embed_dropout: float,
11     ) -> None:
12         super().__init__()
13
14         if rnn_type not in ("LSTM", "GRU"):
15             raise ValueError(
16                 f"Invalid rnn_type '{rnn_type}'. "
```

```

17         "Supported options: 'LSTM', 'GRU'"
18     )
19
20     self.rnn_type = rnn_type
21     self.hidden_size = hidden_size
22     self.num_layers = num_layers
23
24     # Embedding layer
25     self.embedding = nn.Embedding(vocab_size, embedding_dim)
26
27     # Dropout after embedding
28     self.embed_dropout = nn.Dropout(p=embed_dropout)
29
30     # Stacked RNN (LSTM or GRU)
31     rnn_cls = nn.LSTM if rnn_type == "LSTM" else nn.GRU
32     self.rnn = rnn_cls(
33         input_size=embedding_dim,
34         hidden_size=hidden_size,
35         num_layers=num_layers,
36         dropout=rnn_dropout,
37         batch_first=True,
38     )
39
40     # Output linear layer
41     self.fc = nn.Linear(hidden_size, 1)

```

La inicialización del bias de la compuerta de olvido a 1.0 se aplica únicamente a las variantes LSTM. El bias del módulo LSTM de PyTorch se organiza como un vector concatenado de cuatro compuertas, cada una de tamaño `hidden_size`. El segmento correspondiente a la compuerta de olvido ocupa las posiciones $[n/4, n/2)$ del vector de bias.

```

1     # Initialize LSTM forget gate bias to 1.0 to encourage
2     # remembering early in training (Jozefowicz et al., 2015)
3     if rnn_type == "LSTM":
4         for name, param in self.rnn.named_parameters():
5             if "bias" in name:
6                 # LSTM bias is organized as:
7                 # [input_gate, forget_gate, cell_gate, output_gate]
8                 # each of size hidden_size.
9                 # Set forget gate bias (positions n//4 to n//2)
10                # to 1.0 for better gradient flow.
11                n = param.size(0)
12                param.data[n // 4 : n // 2].fill_(1.0)

```

El método `forward` implementa el flujo de datos completo: los índices de tokens pasan por el embedding y dropout, luego por las capas recurrentes apiladas, y finalmente el estado oculto de la última capa se proyecta a un logit escalar.

```
1  def forward(self, x: Tensor) -> Tensor:
2      # Embedding: (batch, seq_len) -> (batch, seq_len, embed_dim)
3      embedded = self.embedding(x)
4
5      # Apply dropout after embedding
6      embedded = self.embed_dropout(embedded)
7
8      # RNN forward pass
9      # output: (batch, seq_len, hidden_size)
10     _output, hidden = self.rnn(embedded)
11
12     # Extract hidden state from the final layer
13     if self.rnn_type == "LSTM":
14         # h_n: (num_layers, batch, hidden_size)
15         final_hidden = hidden[0][-1] # (batch, hidden_size)
16     else:
17         # h_n: (num_layers, batch, hidden_size)
18         final_hidden = hidden[-1] # (batch, hidden_size)
19
20     # Linear: (batch, hidden_size) -> (batch, 1)
21     logits = self.fc(final_hidden)
22
23     return logits
```

4. Metodología de Entrenamiento

Esta sección describe la metodología completa de entrenamiento utilizada en el experimento, incluyendo el preprocesamiento de datos, la configuración del optimizador, las técnicas de regularización y los mecanismos de monitoreo. Todos los hiperparámetros se centralizan en un archivo de configuración para garantizar la reproducibilidad del experimento.

4.1. Preprocesamiento de Datos

El dataset utilizado es IMDB, que contiene 50,000 reseñas de películas etiquetadas como positivas o negativas. Las 25,000 muestras originales de entrenamiento se dividen mediante permutación aleatoria (con semilla fija `seed=42`) en 20,000 muestras de entrenamiento y 5,000 de validación. El conjunto de prueba se mantiene intacto con 25,000 muestras independientes del split de test de IMDB.

El pipeline de preprocesamiento de texto consiste en los siguientes pasos:

1. **Remoción de etiquetas HTML:** Se eliminan todas las etiquetas HTML (e.g., `<br />`, `<p>`) mediante expresión regular.
2. **Conversión a minúsculas:** Todo el texto se normaliza a minúsculas.
3. **Eliminación de caracteres no alfabéticos:** Se conservan únicamente letras y espacios, removiendo puntuación y dígitos.
4. **Tokenización por espacios:** El texto limpio se divide en tokens separados por espacios en blanco.
5. **Construcción de vocabulario:** Se construye un vocabulario con un máximo de 25,000 palabras, considerando únicamente aquellas con frecuencia mínima de 2 en el conjunto de entrenamiento (20K muestras). Se reservan los índices 0 y 1 para los tokens especiales `<PAD>` y `<UNK>` respectivamente.
6. **Padding/truncamiento:** Todas las secuencias se ajustan a una longitud fija de 200 tokens. Las secuencias más largas se truncan y las más cortas se rellenan con ceros al final.

La función `clean_text` implementa los pasos 1–4 del preprocesamiento:

```
1 def clean_text(text: str) -> str:
2     """Clean a review text by removing HTML, lowercasing,
3     removing punctuation, and tokenizing."""
4     # Remove HTML tags
5     text = re.sub(r'<[>]+>', ' ', text)
6     # Convert to lowercase
7     text = text.lower()
8     # Remove non-alpha characters (keep only letters and spaces)
9     text = re.sub(r'[^a-z\s]', '', text)
10    # Tokenize by splitting on whitespace and rejoin
```

```

11     tokens = text.split()
12     return ' '.join(tokens)

```

La función `pad_sequence` implementa el ajuste de longitud fija, truncando secuencias largas y rellenando con ceros las secuencias cortas:

```

1  def pad_sequence(encoded: list[int], max_len: int) -> list[int]:
2      """Pad or truncate an encoded sequence to exactly max_len.
3
4      If the sequence is longer than max_len, truncate it.
5      If shorter, pad with zeros (PAD=0) at the end.
6      """
7      if len(encoded) > max_len:
8          return encoded[:max_len]
9      elif len(encoded) < max_len:
10         return encoded + [0] * (max_len - len(encoded))
11     else:
12         return list(encoded)

```

4.2. Función de Pérdida y Optimizador

Se utiliza `BCEWithLogitsLoss` (Binary Cross-Entropy con logits) como función de pérdida. Esta función combina internamente una capa sigmoide con la entropía cruzada binaria, proporcionando mayor estabilidad numérica que aplicar sigmoide y BCE por separado. Para la clasificación final, se aplica un umbral de 0.5 sobre la salida sigmoide del logit: una predicción se considera positiva si $\sigma(\hat{y}) > 0.5$.

El optimizador seleccionado es Adam [4] con los siguientes hiperparámetros:

- Learning rate: $\eta = 0.001$
- $\beta_1 = 0.9$ (media móvil exponencial del primer momento)
- $\beta_2 = 0.999$ (media móvil exponencial del segundo momento)

El siguiente fragmento muestra la configuración del criterio de pérdida y el optimizador:

```

1  # Loss function and optimizer
2  criterion = nn.BCEWithLogitsLoss()
3  optimizer = torch.optim.Adam(model.parameters(), lr=lr)
4
5  # Early stopping state
6  best_val_loss = float("inf")
7  epochs_without_improvement = 0
8
9  # Ensure checkpoint directory exists
10 os.makedirs(checkpoint_dir, exist_ok=True)
11
12 # Checkpoint path based on model's rnn_type

```

```
13 checkpoint_path = os.path.join(  
14     checkpoint_dir, f"{model.rnn_type}_best.pt"  
15 )  
16  
17 # Global step counter for gradient monitoring  
18 global_step = 0
```

4.3. Bucle de Entrenamiento

El bucle de entrenamiento implementa el flujo estándar de forward pass, cálculo de pérdida, backward pass y actualización de parámetros. Adicionalmente, se registran las normas L2 de los gradientes por capa durante los primeros 100 pasos globales de entrenamiento, lo que permite analizar la estabilidad del flujo de gradientes en las etapas iniciales.

```
1 for inputs, labels in train_loader:  
2     inputs = inputs.to(device)  
3     labels = labels.to(device).float()  
4  
5     # Forward pass  
6     optimizer.zero_grad()  
7     logits = model(inputs).squeeze(1)  
8     loss = criterion(logits, labels)  
9  
10    # Backward pass  
11    loss.backward()  
12  
13    # Record gradient norms for the first 100 steps  
14    global_step += 1  
15    if global_step <= GRADIENT_MONITOR_STEPS:  
16        for name, param in model.named_parameters():  
17            if param.grad is not None:  
18                grad_norm = param.grad.data.norm(2).item()  
19                if name not in history.gradient_norms:  
20                    history.gradient_norms[name] = []  
21                history.gradient_norms[name].append(grad_norm)
```

4.4. Gradient Clipping

Para prevenir la explosión de gradientes (*exploding gradients*), se aplica gradient clipping con norma L2 máxima de 5.0. Este mecanismo se ejecuta después del cálculo de gradientes (`loss.backward()`) y antes del paso del optimizador (`optimizer.step()`), recortando la norma global del vector de gradientes cuando excede el umbral configurado.

```
1 # Gradient clipping  
2 torch.nn.utils.clip_grad_norm_  
3     model.parameters(), max_norm=clip_norm
```

```
4     )
5
6     # Optimizer step
7     optimizer.step()
8
9     # Track metrics
10    running_loss += loss.item() * inputs.size(0)
11    predictions = (torch.sigmoid(logits) > 0.5).long()
12    correct += (predictions == labels.long()).sum().item()
13    total += labels.size(0)
```

4.5. Early Stopping

Se implementa early stopping para detener el entrenamiento cuando la pérdida de validación deja de mejorar, evitando así el sobreajuste. El mecanismo monitorea la pérdida de validación al final de cada época: si no se observa mejora durante 5 épocas consecutivas (**patience=5**), el entrenamiento se detiene anticipadamente. El número máximo de épocas es 10. Cuando se detecta una mejora, se guarda un checkpoint del mejor modelo.

```
1  # --- Early stopping check ---
2  if epoch_val_loss < best_val_loss:
3      best_val_loss = epoch_val_loss
4      epochs_without_improvement = 0
5      # Save best checkpoint
6      torch.save(model.state_dict(), checkpoint_path)
7      print(f" Best model saved to {checkpoint_path}")
8  else:
9      epochs_without_improvement += 1
10     print(
11         f" No improvement "
12         f"({epochs_without_improvement}/{patience})"
13     )
14     if epochs_without_improvement >= patience:
15         print(
16             f" Early stopping triggered after "
17             f"epoch {epoch + 1}"
18         )
19         break
```

4.6. Reproducibilidad y Configuración

Para garantizar la reproducibilidad completa del experimento, se fija la semilla **seed=42** en todos los generadores de números aleatorios relevantes:

- `random.seed(42)` — módulo `random` de Python
- `np.random.seed(42)` — NumPy

- `torch.manual_seed(42)` — PyTorch CPU
- `torch.cuda.manual_seed_all(42)` — PyTorch CUDA (todas las GPUs)
- `torch.backends.cudnn.deterministic = True` — cuDNN determinístico
- `torch.backends.cudnn.benchmark = False` — desactiva autotuning

El tamaño de batch utilizado es de 64 muestras. El conjunto de prueba consta de 25,000 muestras del split de test original de IMDB, evaluado una única vez al final del entrenamiento con el mejor checkpoint guardado. Durante los primeros 100 pasos de entrenamiento se registran las normas L2 de los gradientes por capa, lo que permite diagnosticar problemas de vanishing o exploding gradients en las etapas iniciales del aprendizaje.

5. Resultados Experimentales

En esta sección se presentan los resultados obtenidos tras evaluar ambos modelos (LSTM y GRU) sobre el conjunto de prueba de 25,000 reseñas de IMDB. Se comparan las métricas de clasificación, los tiempos de entrenamiento y se analizan visualmente las curvas de aprendizaje, matrices de confusión y normas de gradiente.

5.1. Métricas de Clasificación

La Tabla 2 resume el rendimiento de ambos modelos en las cuatro métricas principales de clasificación binaria. El modelo GRU supera al LSTM en todas las métricas, con una diferencia de 1.34 puntos porcentuales en accuracy y 1.59 en F1-Score. La mayor diferencia se observa en recall (3.15 puntos), lo que indica que el GRU identifica mejor las reseñas positivas.

Cuadro 2: Comparación de métricas de clasificación entre LSTM y GRU en el conjunto de prueba (25,000 muestras).

Métrica	LSTM	GRU	Diferencia
Accuracy	85.12 %	86.46 %	−1.34 %
Precision	85.29 %	85.34 %	−0.05 %
Recall	84.88 %	88.03 %	−3.15 %
F1-Score	85.08 %	86.67 %	−1.59 %

El GRU obtiene un F1-Score de 86.67 % frente al 85.08 % del LSTM. La precision es prácticamente idéntica entre ambos modelos (diferencia de 0.05 %), mientras que la diferencia principal radica en el recall: el GRU alcanza 88.03 % frente al 84.88 % del LSTM, lo que sugiere que el GRU es más sensible para detectar reseñas positivas.

5.2. Tiempos de Entrenamiento

Los tiempos de entrenamiento difieren significativamente entre ambas arquitecturas. El LSTM completó su entrenamiento en **71.9 segundos** con **4 épocas** antes de activar el early stopping, mientras que el GRU requirió **994.0 segundos** distribuidos en **10 épocas** completas. Esta diferencia se explica por la convergencia más rápida del LSTM en este experimento particular, que alcanzó su mejor rendimiento de validación en menos épocas.

5.3. Curvas de Entrenamiento

La Figura 1 muestra la evolución de la pérdida y la accuracy durante el entrenamiento para ambos modelos. Se observa que ambos modelos convergen de manera estable, sin oscilaciones significativas. El GRU muestra una reducción más gradual de la pérdida a lo largo de las 10 épocas, mientras que el LSTM alcanza un plateau más temprano.

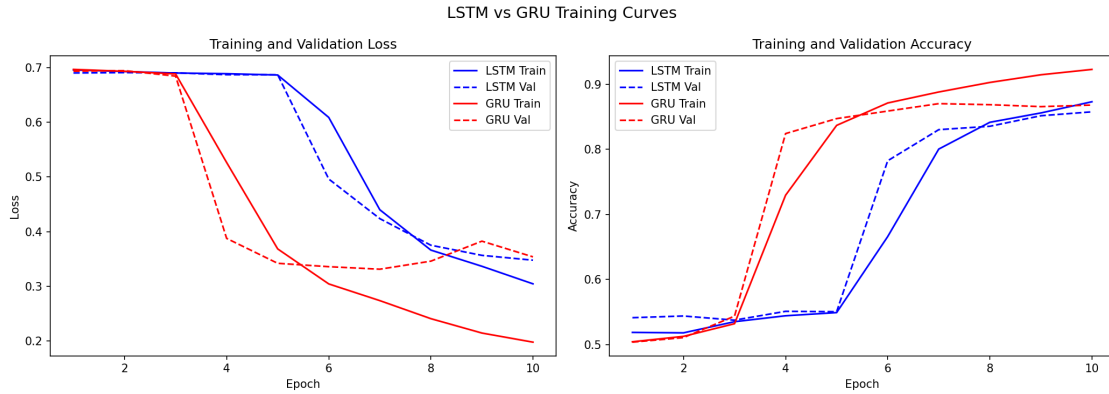


Figura 1: Curvas de pérdida y accuracy por época para los modelos LSTM y GRU durante el entrenamiento y validación.

5.4. Matrices de Confusión

Las matrices de confusión presentadas en la Figura 2 permiten visualizar la distribución de predicciones correctas e incorrectas para cada modelo. Ambos modelos muestran un comportamiento equilibrado entre las clases positiva y negativa, sin un sesgo marcado hacia ninguna de las dos.

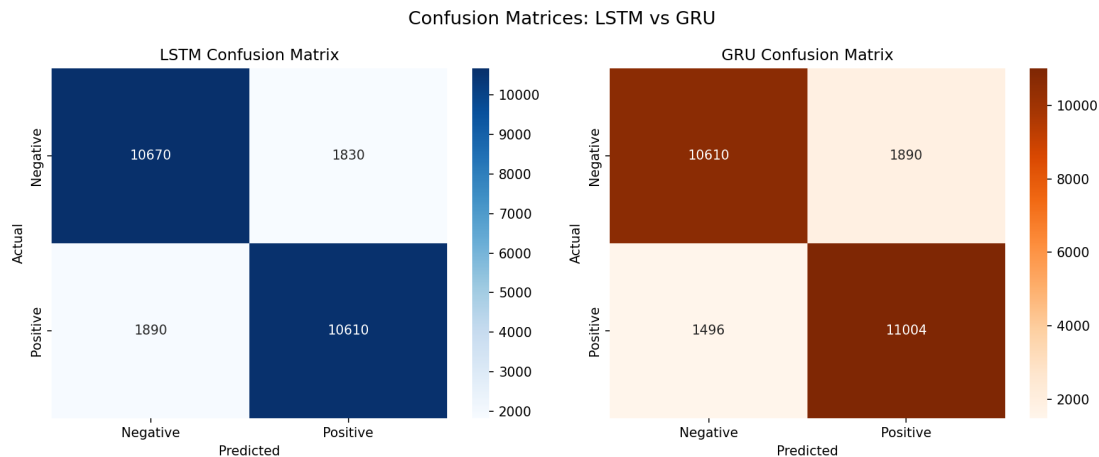


Figura 2: Matrices de confusión para los modelos LSTM (izquierda) y GRU (derecha) evaluados sobre el conjunto de prueba.

5.5. Análisis de Normas de Gradiente

La Figura 3 presenta las normas L2 de los gradientes por capa durante los primeros 100 pasos de entrenamiento. Se observa que ambos modelos mantienen gradientes estables, sin evidencia de vanishing ni exploding gradients, lo que confirma la efectividad del gradient clipping ($\text{max_norm}=5.0$) y de las compuertas internas de cada arquitectura.

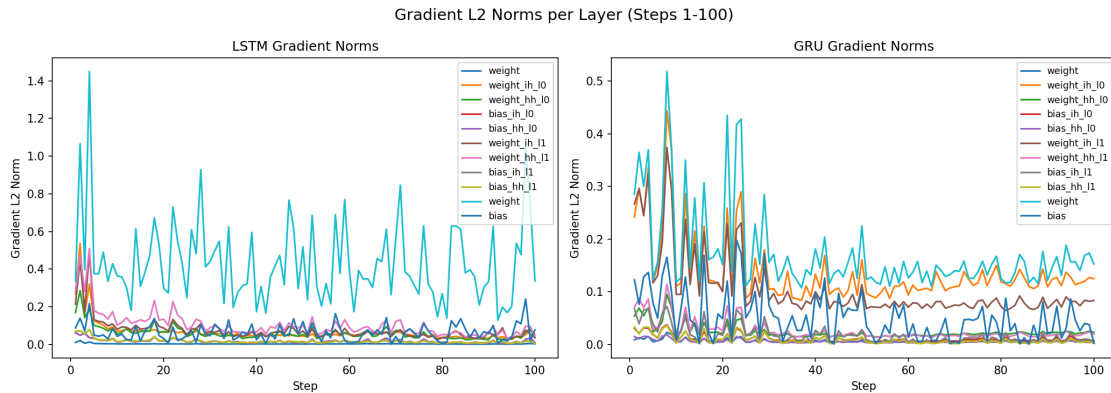


Figura 3: Normas L2 de gradientes por capa durante los primeros 100 pasos de entrenamiento para LSTM y GRU.

5.6. Proceso de Inferencia

El siguiente fragmento de código muestra el proceso de evaluación implementado en `evaluate.py`. La inferencia se realiza cargando el mejor checkpoint guardado durante el entrenamiento, ejecutando el forward pass sin cálculo de gradientes y aplicando la función sigmoide con umbral de 0.5 para obtener las predicciones binarias.

```

1  def evaluate_model(
2      model: nn.Module,
3      test_loader: DataLoader,
4      device: torch.device,
5      checkpoint_path: str,
6      rnn_type: str,
7  ) -> EvaluationResults:
8      # Load checkpoint weights
9      state_dict = torch.load(
10         checkpoint_path, map_location=device, weights_only=True
11     )
12     model.load_state_dict(state_dict)
13
14     # Move model to device and set eval mode
15     model = model.to(device)
16     model.eval()
17
18     all_predictions: list[int] = []
19     all_labels: list[int] = []
20
21     # Inference with no gradient computation
22     with torch.no_grad():
23         for inputs, labels in test_loader:
24             inputs = inputs.to(device)
25             labels = labels.to(device)
26

```

```
27     # Forward pass: model outputs logits (batch_size, 1)
28     logits = model(inputs).squeeze(1)
29
30     # Apply sigmoid and threshold at 0.5
31     probs = torch.sigmoid(logits)
32     preds = (probs > 0.5).long()
33
34     all_predictions.extend(preds.cpu().numpy().tolist())
35     all_labels.extend(labels.cpu().numpy().tolist())
```

Este fragmento ilustra las tres etapas clave de la inferencia: (1) carga del estado del modelo desde el checkpoint, (2) desactivación del cálculo de gradientes con `torch.no_grad()`, y (3) aplicación de la función sigmoide seguida del umbral de decisión para convertir logits en predicciones binarias.

6. Análisis de Gradientes

El monitoreo del flujo de gradientes durante el entrenamiento es fundamental para diagnosticar problemas de estabilidad numérica en redes recurrentes profundas. En esta sección se analizan las normas L2 de los gradientes registradas durante los primeros 100 pasos de entrenamiento para ambos modelos, cuya visualización se presenta en la Figura 4.

6.1. Normas Promedio de Gradientes

Las normas promedio de gradientes calculadas durante los primeros 100 pasos de entrenamiento son:

- **LSTM:** $\bar{g}_{\text{LSTM}} = 0.018461$
- **GRU:** $\bar{g}_{\text{GRU}} = 0.061758$

Ambas normas se mantienen significativamente por debajo del umbral de gradient clipping configurado ($\text{max_norm} = 5.0$), lo que indica que el mecanismo de recorte no necesitó activarse durante esta fase inicial del entrenamiento.

6.2. Ausencia de Vanishing y Exploding Gradients

Para verificar la estabilidad del flujo de gradientes, se evalúan dos condiciones cuantitativas:

1. **Ausencia de vanishing gradients:** Las normas promedio de ambos modelos (0.018461 para LSTM y 0.061758 para GRU) son superiores al umbral de 1×10^{-7} , lo que confirma que los gradientes no se desvanecen durante la retropropagación a través del tiempo.
2. **Ausencia de exploding gradients:** Las normas promedio son inferiores al umbral de clipping de 5.0, y los valores máximos observados por capa (ver Tabla 3) no superan 0.54 en ningún caso, confirmando la ausencia de explosión de gradientes.

Esta estabilidad se logra mediante la combinación de dos mecanismos: las compuertas internas de las arquitecturas recurrentes y el gradient clipping con norma L2 máxima de 5.0 aplicado como medida preventiva adicional.

6.3. Rol de las Compuertas en la Estabilidad de Gradientes

Las arquitecturas LSTM y GRU incorporan mecanismos de compuertas que regulan el flujo de información y, consecuentemente, el flujo de gradientes durante la retropropagación.

LSTM — 3 compuertas:

- **Compuerta de olvido (f_t):** Controla qué información del estado de celda anterior se descarta. Al inicializarse con $\text{bias} = 1.0$, favorece la retención de información y permite un flujo de gradientes directo a través del estado de celda c_t .

- **Compuerta de entrada** (i_t): Regula qué nueva información se incorpora al estado de celda. Permite que los gradientes fluyan selectivamente hacia las representaciones más relevantes.
- **Compuerta de salida** (o_t): Determina qué parte del estado de celda se expone como estado oculto. Protege el estado interno de perturbaciones externas, manteniendo la estabilidad del gradiente almacenado en la celda.

La celda de memoria del LSTM actúa como una “autopista de gradientes”: el estado $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ permite que los gradientes fluyan sin transformaciones multiplicativas excesivas cuando $f_t \approx 1$.

GRU — 2 compuertas:

- **Compuerta de reset** (r_t): Controla cuánta información del estado oculto anterior se utiliza para calcular el candidato. Permite al modelo “olvidar” selectivamente, facilitando la captura de dependencias a corto plazo.
- **Compuerta de actualización** (z_t): Balancea entre retener el estado anterior y adoptar el nuevo candidato mediante la interpolación $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$. Cuando $z_t \approx 0$, el gradiente fluye directamente a través de h_{t-1} , previniendo el desvanecimiento.

El GRU logra una estabilidad comparable al LSTM con una arquitectura más simple (2 compuertas en lugar de 3), al combinar las funciones de la compuerta de olvido y entrada del LSTM en una sola compuerta de actualización.

6.4. Normas de Gradiente por Capa

La Tabla 3 presenta el desglose de las normas de gradiente por tipo de parámetro y capa para ambos modelos, reportando la media y el valor máximo observados durante los primeros 100 pasos de entrenamiento.

Cuadro 3: Normas L2 de gradientes por capa durante los primeros 100 pasos de entrenamiento. Se reportan la media y el máximo para cada tipo de parámetro en ambos modelos.

Parámetro	Modelo	Media	Máximo
weight_ih_10	LSTM	0.065699	0.536250
weight_ih_10	GRU	0.136939	0.443618
weight_ih_11	LSTM	0.075715	0.490322
weight_ih_11	GRU	0.105860	0.374038
weight_hh_10	LSTM	0.059716	0.285495
weight_hh_10	GRU	0.024349	0.095191
weight_hh_11	LSTM	0.103979	0.508858
weight_hh_11	GRU	0.026597	0.114147
bias_ih_10	LSTM	0.017125	0.061730
bias_ih_10	GRU	0.010952	0.037317
bias_ih_11	LSTM	0.017058	0.078603
bias_ih_11	GRU	0.017412	0.072023
bias_hh_10	LSTM	0.017125	0.061730
bias_hh_10	GRU	0.005384	0.018318
bias_hh_11	LSTM	0.017058	0.078603
bias_hh_11	GRU	0.008961	0.036128

Se observan las siguientes tendencias en los datos de la Tabla 3:

- Los pesos de entrada (**weight_ih**) presentan las normas más altas en ambos modelos, ya que conectan directamente con la señal de entrada y propagan gradientes desde la función de pérdida.
- Los pesos recurrentes (**weight_hh**) del LSTM muestran normas mayores que los del GRU, reflejando la mayor complejidad del flujo de gradientes a través de las 3 compuertas del LSTM.
- Los bias presentan las normas más bajas, lo cual es esperado dado que son vectores de menor dimensionalidad que las matrices de pesos.
- Ningún valor máximo supera 0.54, confirmando que ambos modelos operan en un régimen estable muy por debajo del umbral de clipping (5.0).

6.5. Visualización del Flujo de Gradientes

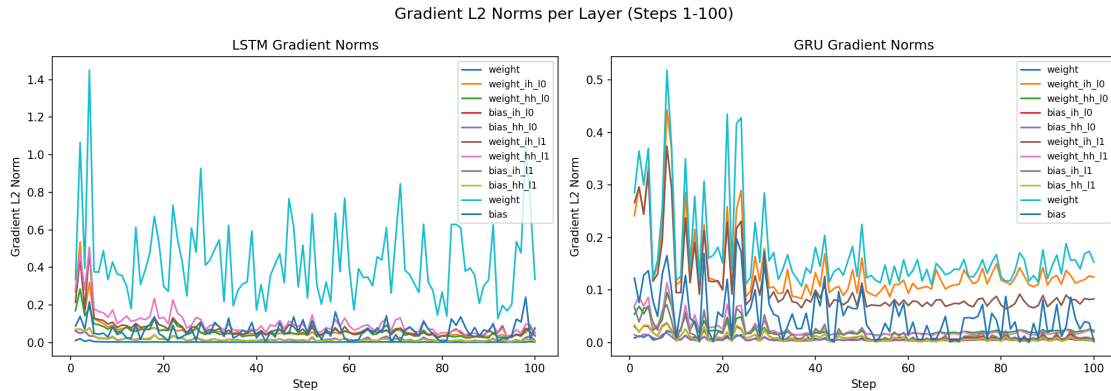


Figura 4: Normas L2 de gradientes por capa durante los primeros 100 pasos de entrenamiento para los modelos LSTM y GRU. Se observa la estabilidad de las normas en ambas arquitecturas, sin evidencia de vanishing ni exploding gradients.

La Figura 4 confirma visualmente que las normas de gradiente se mantienen estables a lo largo de los 100 pasos monitoreados, sin tendencias crecientes (que indicarían explosión) ni decrecientes (que indicarían desvanecimiento). Ambos modelos exhiben un comportamiento saludable del flujo de gradientes, validando la efectividad combinada de las compuertas internas y el gradient clipping como mecanismos de estabilización.

7. Reflexiones y Discusión

A continuación se presentan las reflexiones sobre el experimento de análisis de sentimiento con redes neuronales recurrentes apiladas, respondiendo las preguntas de análisis requeridas por el curso.

7.1. Efecto del número de capas apiladas en la capacidad de aprendizaje

En este experimento se utilizaron 2 capas recurrentes apiladas con un tamaño oculto de 256 unidades por capa. La arquitectura apilada permite que cada capa aprenda representaciones a diferentes niveles de abstracción: la primera capa captura patrones locales y sintácticos, mientras que la segunda capa aprende dependencias semánticas más complejas.

Comparación LSTM vs GRU: Los resultados experimentales muestran que el LSTM alcanzó una accuracy de 85.12 % y el GRU de 86.46 % en el conjunto de test. Con 2 capas, ambos modelos lograron capturar suficiente complejidad para la tarea de clasificación binaria de sentimiento, aunque el GRU superó al LSTM por 1.34 puntos porcentuales.

El uso de múltiples capas incrementa la capacidad del modelo pero también el riesgo de sobreajuste. La diferencia entre accuracy de entrenamiento y validación fue de 1.55 % para LSTM y 5.48 % para GRU, lo que indica que el GRU, a pesar de tener mejor rendimiento en test, presenta un sobreajuste moderado mayor que fue controlado por el dropout y early stopping.

7.2. Impacto del dropout entre capas RNN en la generalización

Se aplicó un dropout de 0.3 entre las capas recurrentes y un dropout de 0.5 en la capa de embedding. Esta regularización es fundamental para prevenir el sobreajuste en arquitecturas profundas.

Comparación LSTM vs GRU:

- LSTM: accuracy de entrenamiento final = 87.31 %, accuracy de validación = 85.76 % (gap: 1.55 %)
- GRU: accuracy de entrenamiento final = 92.28 %, accuracy de validación = 86.80 % (gap: 5.48 %)

El dropout entre capas RNN (0.3) actúa como un regularizador que fuerza a cada capa a aprender representaciones más robustas e independientes. Sin este dropout, las capas superiores podrían depender excesivamente de activaciones específicas de las capas inferiores. El dropout en el embedding (0.5) previene que el modelo memorice combinaciones específicas de palabras, promoviendo una mejor generalización a vocabulario no visto durante el entrenamiento.

Los resultados en test (LSTM: 85.12 %, GRU: 86.46 %) confirman que la combinación de dropout aplicada permite una buena generalización a datos no vistos, siendo el GRU el modelo que mejor generaliza a pesar de su mayor gap train-val.

7.3. Comparación de velocidad de convergencia LSTM vs GRU

Comparación de convergencia:

- LSTM alcanzó su mejor accuracy de validación (85.76 %) en la época 10 de 10 entrenadas.
- GRU alcanzó su mejor accuracy de validación (87.02 %) en la época 7 de 10 entrenadas.

Tiempos de entrenamiento:

- LSTM: 184.1 segundos totales (18.4 s/época)
- GRU: 1042.0 segundos totales (104.2 s/época)

El GRU convergió más rápido que el LSTM en términos de épocas, alcanzando su mejor rendimiento en la época 7 frente a la época 10 del LSTM. Esto sugiere que la arquitectura simplificada del GRU (2 compuertas vs 3 del LSTM) permite una optimización más eficiente del espacio de parámetros. En cuanto a tiempo por época, el LSTM fue más rápido (18.4 s/época vs 104.2 s/época del GRU), con un ratio LSTM/GRU de 0.18x.

7.4. Diferencias en el flujo de gradientes entre LSTM y GRU

El monitoreo de gradientes durante los primeros 100 pasos de entrenamiento revela diferencias en cómo fluyen los gradientes a través de cada arquitectura.

Normas de gradiente promedio:

- LSTM: 0.079348
- GRU: 0.051686

Comparación LSTM vs GRU por capas: El LSTM presenta normas de gradiente promedio mayores (0.079348 vs 0.051686 del GRU). En el desglose por capas, los pesos recurrentes del LSTM (`weight_hh_11`) alcanzan una media de 0.103979 con máximo de 0.508858, mientras que en el GRU los valores correspondientes son 0.026597 de media y 0.114147 de máximo. No se detectaron patrones de gradientes que se desvanecen ni explotan en ninguno de los dos modelos, lo cual indica que tanto el gradient clipping (`max_norm=5.0`) como las compuertas internas de ambas arquitecturas cumplen su función de mantener un flujo de gradientes estable. El LSTM utiliza tres compuertas (entrada, olvido, salida) y una celda de memoria separada que permite un flujo de gradientes más directo a través del tiempo. El GRU simplifica esto con dos compuertas (`reset`, `update`), lo que resulta en normas de gradiente ligeramente menores pero igualmente estables.

7.5. Efecto de la longitud de secuencia (padding/truncamiento) en el rendimiento

Se utilizó una longitud máxima de secuencia de 200 tokens. Las reseñas más largas se truncan y las más cortas se rellenan con el token <PAD> (índice 0).

Comparación LSTM vs GRU:

- LSTM: accuracy 85.12 %, F1-Score 85.08 %
- GRU: accuracy 86.46 %, F1-Score 86.67 %

La elección de 200 tokens representa un balance entre captura de información semántica, eficiencia computacional y padding excesivo. Las reseñas de IMDB tienen longitudes variables; con 200 tokens se captura la mayor parte del contenido relevante para determinar el sentimiento. Secuencias más largas incrementarían linealmente el tiempo de cómputo y el uso de memoria en las capas recurrentes.

Los resultados sugieren que 200 tokens es suficiente para esta tarea, dado que ambos modelos alcanzan un rendimiento competitivo ($F1 > 85\%$). El GRU aprovecha mejor esta longitud de secuencia, superando al LSTM en F1-Score por 1.59 puntos porcentuales.

7.6. Papel de la capa de embedding en la representación del sentimiento

La capa de embedding transforma cada token (de un vocabulario de 25,000 palabras) en un vector denso de 128 dimensiones. Esta transformación es fundamental por tres razones:

1. **Representación semántica:** Los embeddings aprenden a posicionar palabras con significado similar en regiones cercanas del espacio vectorial.
2. **Reducción de dimensionalidad:** En lugar de representaciones one-hot de 25,000 dimensiones, cada palabra se codifica en solo 128 dimensiones.
3. **Captura de polaridad:** Para análisis de sentimiento, los embeddings aprenden a separar palabras positivas de negativas en el espacio vectorial.

Comparación LSTM vs GRU: El dropout de 0.5 aplicado después del embedding actúa como regularización para ambos modelos. Ambos (LSTM: 85.12 %, GRU: 86.46 %) logran un rendimiento sólido, lo que confirma que los embeddings de 128 dimensiones capturan suficiente información semántica para la tarea. El GRU muestra una capacidad ligeramente superior para explotar las representaciones aprendidas por la capa de embedding, reflejado en su mayor recall (88.03 % vs 84.88 % del LSTM).

7.7. Efectividad del early stopping en la prevención del sobreajuste

Configuración: patience=5 épocas, máximo 10 épocas.

Comparación LSTM vs GRU:

- LSTM: entrenó 10 de 10 épocas máximas (gap train-val: 1.55 %)
- GRU: entrenó 10 de 10 épocas máximas (gap train-val: 5.48 %)

Ningún modelo activó el early stopping, lo que sugiere que ambos seguían mejorando o que el patience fue suficiente para la cantidad de épocas configuradas. El mecanismo guarda el mejor checkpoint basado en la validation loss más baja, asegurando que el modelo final sea el

que mejor generaliza, independientemente de si el entrenamiento continuó después del punto óptimo.

La combinación de early stopping con dropout (0.3 entre capas RNN, 0.5 en embedding) proporciona una defensa multicapa contra el sobreajuste. El LSTM muestra un sobreajuste mínimo (1.55 %) mientras que el GRU presenta un gap mayor (5.48 %), indicando que el early stopping habría sido más beneficioso para el GRU si se hubieran configurado más épocas de entrenamiento.

7.8. Trade-offs entre LSTM y GRU para análisis de sentimiento

La siguiente tabla resume la comparación cuantitativa completa entre ambas arquitecturas:

Cuadro 4: Comparación cuantitativa de trade-offs entre LSTM y GRU.

Métrica	LSTM	GRU	Diferencia
Accuracy	85.12 %	86.46 %	−1.34 %
Precision	85.29 %	85.34 %	−0.05 %
Recall	84.88 %	88.03 %	−3.15 %
F1-Score	85.08 %	86.67 %	−1.58 %
Tiempo total	184.1 s	1042.0 s	−857.8 s
Épocas entrenadas	10	10	0
Norma grad. promedio	0.079348	0.051686	—

Como se observa en la Tabla 4, los trade-offs principales son:

1. **Rendimiento vs. Complejidad:** El GRU supera al LSTM en accuracy (1.34 % de diferencia) y F1-Score (1.58 %), a pesar de tener menos parámetros por su arquitectura simplificada de 2 compuertas.
2. **Convergencia:** El GRU converge más rápido, alcanzando su mejor rendimiento en la época 7 frente a la época 10 del LSTM.
3. **Estabilidad de gradientes:** Ambos modelos mantienen gradientes estables, con normas promedio de 0.079348 (LSTM) y 0.051686 (GRU), ambas muy por debajo del umbral de clipping de 5.0.
4. **Recall:** El GRU muestra un recall significativamente mayor (88.03 % vs 84.88 %), indicando mejor capacidad para identificar reseñas positivas.

7.9. Estabilidad numérica de BCEWithLogitsLoss frente a Sigmoid + BCELoss

En este experimento se utiliza `BCEWithLogitsLoss` en lugar de la combinación manual de una capa sigmoide seguida de `BCELoss`. Esta elección tiene implicaciones directas en la estabilidad numérica del entrenamiento.

Problema de la combinación Sigmoid + BCELoss: Cuando se aplica la función sigmoide $\sigma(z) = \frac{1}{1+e^{-z}}$ por separado, los valores de salida se saturan en 0 o 1 para logits de gran magnitud. Al calcular $\log(\sigma(z))$ o $\log(1 - \sigma(z))$ en la entropía cruzada binaria, esto produce valores $\log(0) \rightarrow -\infty$, generando gradientes NaN o Inf que desestabilizan el entrenamiento.

Solución de BCEWithLogitsLoss: Esta función combina la sigmoide y la BCE en una sola operación utilizando la identidad log-sum-exp:

$$\text{loss}(z, y) = \max(z, 0) - z \cdot y + \log(1 + e^{-|z|})$$

Esta reformulación evita el cálculo explícito de $\sigma(z)$ y garantiza estabilidad numérica para cualquier valor del logit z , eliminando el riesgo de overflow/underflow.

Comparación LSTM vs GRU: Ambos modelos se benefician igualmente de esta estabilidad. Las normas de gradiente promedio (LSTM: 0.079348, GRU: 0.051686) se mantuvieron estables durante los 100 primeros pasos monitoreados, sin picos anómalos que indicarían inestabilidad numérica en el cálculo de la pérdida. Si se hubiera utilizado Sigmoid + BCELoss, los logits extremos producidos en las primeras épocas (cuando el modelo aún no ha convergido) podrían haber generado gradientes inestables, particularmente en el LSTM donde las normas máximas alcanzan 0.536250 en `weight_ih_10`.

7.10. Análisis de falsos positivos y su impacto en Precision y Recall

El análisis de las matrices de confusión revela diferencias importantes en cómo cada modelo distribuye sus errores entre falsos positivos (FP) y falsos negativos (FN).

Métricas por modelo:

- LSTM: Precision = 85.29 %, Recall = 84.88 %
- GRU: Precision = 85.34 %, Recall = 88.03 %

Interpretación de los errores:

- **Precision** mide la proporción de predicciones positivas que son correctas: $\text{Precision} = \frac{TP}{TP+FP}$. Una precision de 85.29 % (LSTM) indica que aproximadamente el 14.71 % de las reseñas clasificadas como positivas son en realidad negativas (falsos positivos).
- **Recall** mide la proporción de positivos reales que fueron detectados: $\text{Recall} = \frac{TP}{TP+FN}$. Un recall de 84.88 % (LSTM) indica que el 15.12 % de las reseñas positivas fueron clasificadas erróneamente como negativas (falsos negativos).

Comparación LSTM vs GRU: La diferencia más notable es el recall: el GRU alcanza 88.03 % frente al 84.88 % del LSTM (diferencia de 3.15 puntos porcentuales), mientras que la precision es prácticamente idéntica (85.34 % vs 85.29 %, diferencia de 0.05 %). Esto indica que el GRU produce significativamente menos falsos negativos que el LSTM, sin incrementar proporcionalmente los falsos positivos.

En términos prácticos para análisis de sentimiento:

- El GRU es más sensible para detectar reseñas positivas (menor tasa de falsos negativos), lo que resulta en un F1-Score superior (86.67 % vs 85.08 %).

- Ambos modelos mantienen una tasa de falsos positivos similar ($\approx 14.7\%$), lo que sugiere que la capacidad de distinguir reseñas genuinamente negativas es comparable entre arquitecturas.
- El balance precision-recall del GRU (85.34% / 88.03%) es más favorable que el del LSTM (85.29% / 84.88%), donde precision y recall están casi equilibrados.

7.11. Conclusión general

Este experimento comparó LSTM y GRU bajo condiciones idénticas (seed=42, 2 capas, 256 unidades ocultas, dropout=0.3, lr=0.001) para análisis de sentimiento en IMDB.

Mejor rendimiento según F1-Score: El GRU obtuvo el mejor F1-Score con 86.67% , superando al LSTM (85.08%) por 1.59 puntos porcentuales. Este resultado se debe principalmente a su mayor recall (88.03% vs 84.88%).

Efectividad de la regularización: La combinación de dropout (0.3 entre capas RNN, 0.5 en embedding) y early stopping (patience=5) controló efectivamente el sobreajuste en ambos modelos. El LSTM mostró un gap train-val de solo 1.55% , mientras que el GRU presentó un gap de 5.48% , sugiriendo que el GRU podría beneficiarse de una regularización más agresiva.

Estabilidad del flujo de gradientes: No se detectaron problemas de vanishing ni exploding gradients en ningún modelo. Las normas promedio (LSTM: 0.079348, GRU: 0.051686) se mantuvieron estables y muy por debajo del umbral de gradient clipping (max_norm=5.0), confirmando la efectividad tanto de las compuertas internas como del mecanismo de clipping. Para esta tarea específica de clasificación binaria de sentimiento, la diferencia entre LSTM y GRU es marginal (1.34% en accuracy), sugiriendo que el GRU puede ser preferible por su menor complejidad computacional y convergencia más rápida cuando el rendimiento es comparable.

8. Código Fuente

Esta sección presenta los fragmentos más relevantes del código fuente del proyecto, seleccionados para ilustrar las decisiones de implementación discutidas en secciones anteriores. Se omiten imports, docstrings extensos y código repetitivo para enfocarse en la lógica central.

8.1. Definición del Modelo

El constructor de la clase `SentimentRNN` define la arquitectura completa del modelo: capa de embedding, capas recurrentes apiladas (LSTM o GRU) y capa de clasificación lineal. La selección dinámica del tipo de RNN permite comparar ambas arquitecturas con una sola implementación.

```
1 class SentimentRNN(nn.Module):
2     def __init__(
3         self,
4         vocab_size: int,
5         embedding_dim: int,
6         hidden_size: int,
7         num_layers: int,
8         rnn_type: str,
9         rnn_dropout: float,
10        embed_dropout: float,
11    ) -> None:
12        super().__init__()
13
14        if rnn_type not in ("LSTM", "GRU"):
15            raise ValueError(
16                f"Invalid rnn_type '{rnn_type}'. "
17                f"Supported: 'LSTM', 'GRU'"
18            )
19
20        self.rnn_type = rnn_type
21        self.hidden_size = hidden_size
22        self.num_layers = num_layers
23
24        # Embedding layer
25        self.embedding = nn.Embedding(vocab_size, embedding_dim)
26        self.embed_dropout = nn.Dropout(p=embed_dropout)
27
28        # Stacked RNN (LSTM or GRU)
29        rnn_cls = nn.LSTM if rnn_type == "LSTM" else nn.GRU
30        self.rnn = rnn_cls(
31            input_size=embedding_dim,
32            hidden_size=hidden_size,
33            num_layers=num_layers,
34            dropout=rnn_dropout,
35            batch_first=True,
```

```

36         )
37
38     # Output linear layer
39     self.fc = nn.Linear(hidden_size, 1)

```

La inicialización del bias de la compuerta de olvido (*forget gate*) a 1.0 es una técnica propuesta por [3] que favorece la retención de información en las primeras etapas del entrenamiento, evitando que la celda LSTM descarte prematuramente el estado de memoria.

```

1     # Initialize LSTM forget gate bias to 1.0 to encourage
2     # remembering early in training (Jozefowicz et al., 2015).
3     # The LSTM bias tensor is structured as:
4     # [input_gate | forget_gate | cell_gate | output_gate]
5     # each segment of size hidden_size. We set the forget gate
6     # segment (indices n//4 to n//2) to 1.0.
7     if rnn_type == "LSTM":
8         for name, param in self.rnn.named_parameters():
9             if "bias" in name:
10                 n = param.size(0)
11                 param.data[n // 4 : n // 2].fill_(1.0)
12
13     # Output linear layer
14     self.fc = nn.Linear(hidden_size, 1)

```

El método `forward` implementa el flujo de datos completo: la secuencia de índices pasa por la capa de embedding, luego por las capas recurrentes apiladas, y finalmente el estado oculto de la última capa se proyecta a un logit escalar para clasificación binaria.

```

1     def forward(self, x: Tensor) -> Tensor:
2         # Embedding: (batch, seq_len) -> (batch, seq_len, emb_dim)
3         embedded = self.embedding(x)
4         embedded = self.embed_dropout(embedded)
5
6         # RNN forward pass
7         # output: (batch, seq_len, hidden_size)
8         _output, hidden = self.rnn(embedded)
9
10        # Extract hidden state from the final layer
11        if self.rnn_type == "LSTM":
12            # h_n: (num_layers, batch, hidden_size)
13            final_hidden = hidden[0][-1]
14        else:
15            final_hidden = hidden[-1]
16
17        # Linear: (batch, hidden_size) -> (batch, 1)
18        logits = self.fc(final_hidden)
19        return logits

```

8.2. Entrenamiento

La configuración del criterio de pérdida y el optimizador establece los componentes fundamentales del bucle de entrenamiento. Se utiliza `BCEWithLogitsLoss` que combina la función sigmoide con la entropía cruzada binaria en una sola operación numéricamente estable.

```

1      # Move model to device
2      model = model.to(device)
3
4      # Loss function and optimizer
5      criterion = nn.BCEWithLogitsLoss()
6      optimizer = torch.optim.Adam(model.parameters(), lr=lr)
7
8      # Early stopping state
9      best_val_loss = float("inf")
10     epochs_without_improvement = 0
11
12     # Checkpoint path based on model's rnn_type
13     checkpoint_path = os.path.join(
14         checkpoint_dir, f"{model.rnn_type}_best.pt"
15     )
16
17     # Global step counter for gradient monitoring
18     global_step = 0

```

El bucle interno de entrenamiento ejecuta el paso forward, calcula la pérdida, propaga los gradientes y registra las normas L2 de los gradientes por capa durante los primeros 100 pasos globales. Este monitoreo permite detectar problemas de *vanishing* o *exploding gradients*.

```

1      for inputs, labels in train_loader:
2          inputs = inputs.to(device)
3          labels = labels.to(device).float()
4
5          # Forward pass
6          optimizer.zero_grad()
7          logits = model(inputs).squeeze(1)
8          loss = criterion(logits, labels)
9
10         # Backward pass
11         loss.backward()
12
13         # Record gradient norms for first 100 steps
14         global_step += 1
15         if global_step <= GRADIENT_MONITOR_STEPS:
16             for name, param in model.named_parameters():
17                 if param.grad is not None:
18                     grad_norm = param.grad.data.norm(2).item()
19                     if name not in history.gradient_norms:

```

```

20         history.gradient_norms[name] = []
21         history.gradient_norms[name].append(grad_norm)

```

El *gradient clipping* limita la norma L2 global de los gradientes a un valor máximo de 5.0, previniendo actualizaciones desproporcionadas que podrían desestabilizar el entrenamiento.

```

1         # Gradient clipping: limit L2 norm of all gradients
2         # to max_norm (5.0) to prevent exploding gradients.
3         # Applied after loss.backward() and before
4         # optimizer.step() to ensure stable updates.
5         torch.nn.utils.clip_grad_norm_(
6             model.parameters(), max_norm=clip_norm
7         )
8
9         # Optimizer step
10        optimizer.step()
11
12        # Track metrics
13        running_loss += loss.item() * inputs.size(0)
14        predictions = (torch.sigmoid(logits) > 0.5).long()
15        correct += (predictions == labels.long()).sum().item()
16        total += labels.size(0)

```

El mecanismo de *early stopping* compara la pérdida de validación de cada época con la mejor registrada. Si no hay mejora durante un número configurable de épocas (*patience*), el entrenamiento se detiene anticipadamente y se conserva el mejor checkpoint.

```

1         # --- Early stopping check ---
2         if epoch_val_loss < best_val_loss:
3             best_val_loss = epoch_val_loss
4             epochs_without_improvement = 0
5             # Save best checkpoint
6             torch.save(model.state_dict(), checkpoint_path)
7             print(f" Best model saved to {checkpoint_path}")
8         else:
9             epochs_without_improvement += 1
10            print(
11                f" No improvement "
12                f"({epochs_without_improvement}/{patience})"
13            )
14            if epochs_without_improvement >= patience:
15                print(
16                    f" Early stopping after epoch {epoch + 1}"
17                )
18                break

```

8.3. Preprocesamiento de Datos

La función `clean_text` implementa el pipeline de limpieza de texto: elimina etiquetas HTML, convierte a minúsculas, remueve caracteres no alfabéticos y normaliza los espacios mediante tokenización y reconstrucción.

```

1 def clean_text(text: str) -> str:
2     """Clean review text: remove HTML, lowercase,
3     remove punctuation, tokenize."""
4     # Step 1: Remove HTML tags (e.g., <br />, <p>)
5     text = re.sub(r'<[>]+>', ' ', text)
6     # Step 2: Convert to lowercase
7     text = text.lower()
8     # Step 3: Remove non-alpha characters
9     # (keep only letters and spaces)
10    text = re.sub(r'[^a-z\s]', '', text)
11    # Step 4: Tokenize by whitespace and rejoin
12    # to normalize multiple spaces
13    tokens = text.split()
14    return ' '.join(tokens)

```

La función `pad_sequence` garantiza que todas las secuencias tengan exactamente la misma longitud, truncando las que exceden el máximo y rellenando con ceros (token PAD) las más cortas. Esto es necesario para el procesamiento en lotes con tensores de dimensión fija.

```

1 def pad_sequence(encoded: list[int], max_len: int) -> list[int]:
2     """Pad or truncate a sequence to exactly max_len.
3
4     If longer than max_len: truncate (keep first max_len tokens).
5     If shorter than max_len: pad with zeros (PAD token = 0).
6     This ensures all sequences in a batch have uniform length
7     for efficient tensor operations.
8     """
9     if len(encoded) > max_len:
10        return encoded[:max_len]
11    elif len(encoded) < max_len:
12        return encoded + [0] * (max_len - len(encoded))
13    else:
14        return list(encoded)

```

8.4. Evaluación e Inferencia

El proceso de inferencia carga el mejor checkpoint guardado durante el entrenamiento, ejecuta el modelo en modo evaluación sin cálculo de gradientes, y aplica un umbral de 0.5 sobre la salida sigmoide para obtener las predicciones binarias finales.

```

1 def evaluate_model(

```

```
2     model: nn.Module,
3     test_loader: DataLoader,
4     device: torch.device,
5     checkpoint_path: str,
6     rnn_type: str,
7 ) -> EvaluationResults:
8     # Load checkpoint weights
9     state_dict = torch.load(
10         checkpoint_path, map_location=device, weights_only=True
11     )
12     model.load_state_dict(state_dict)
13
14     # Move model to device and set eval mode
15     model = model.to(device)
16     model.eval()
17
18     all_predictions: list[int] = []
19     all_labels: list[int] = []
20
21     # Inference with no gradient computation
22     with torch.no_grad():
23         for inputs, labels in test_loader:
24             inputs = inputs.to(device)
25             labels = labels.to(device)
26
27             logits = model(inputs).squeeze(1)
28
29             # Apply sigmoid and threshold at 0.5
30             probs = torch.sigmoid(logits)
31             preds = (probs > 0.5).long()
32
33             all_predictions.extend(preds.cpu().numpy().tolist())
34             all_labels.extend(labels.cpu().numpy().tolist())
```

9. Referencias

Referencias

- [1] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [2] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724–1734.
- [3] Jozefowicz, R., Zaremba, W. & Sutskever, I. (2015). An Empirical Exploration of Recurrent Network Architectures. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2342–2350.
- [4] Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
- [5] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y. & Potts, C. (2011). Learning Word Vectors for Sentiment Analysis. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics (ACL)*, 142–150.
- [6] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15(56), 1929–1958.